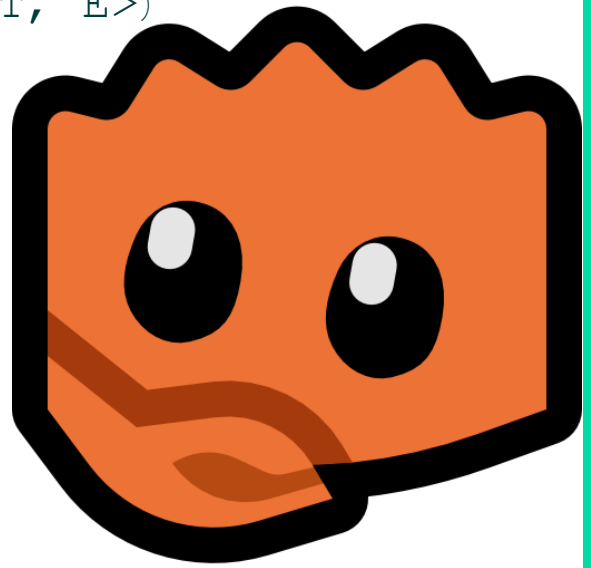
University of Exeter

ECM2433: The C Family

Lecture 05: Error Handling and File I/O

Rust's error philosophy

- Two categories of errors in Rust
- Unrecoverable: bug in the program, stop immediately (`panic!`)
- Recoverable: expected failure, caller decides what to do (`Result<T, E>`)
- Rust makes error handling explicit and compiler-checked
- [Book ch 9.0]



University of Exeter

`panic!` and when to use it

```rust
fn divide(a: i32, b: i32) -> i32 {
    if b == 0 {
        panic!("division by zero");
    }
    a / b
}

fn main() {
    let result = divide(10, 0);
    println!("{result}");
}
```

- `panic!` immediately stops the program with an error message
- Used for unrecoverable errors: bugs, violated assumptions
- Some operations panic automatically (e.g., out-of-bounds indexing)
- [Book ch 9.1]

```
[gad212@mcrugcomp01 hello-rust]$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `target/debug/hello-rust`

thread 'main' panicked at src/main.rs:3:9:
division by zero
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

University of Exeter

Introducing `Result<T, E>`

```
1 enum Result<T, E> {
2     Ok(T),
3     Err(E),
4 }
```

- `Result<T, E>`: an enum with two variants
- `Ok(T)` : operation succeeded, holds the value
- `Err(E)` : operation failed, holds the error
- Like `Option`, but the "nothing" case carries error information
- [Book ch 9.2]

University of Exeter

Handling `Result` with `match`

```rust
fn main() {
    let input = "42";
    let number: Result<i32, _> = input.parse();

    match number {
        Ok(n) => println!("parsed: {n}"),    // parsed: 42
        Err(e) => println!("failed: {e}"),
    }
}
```

- `parse()` returns `Result<T, E>`, not the value directly
- `match` extracts the value from `Ok` or handles the `Err`
- Same pattern as matching on `Option`, but with error information
- [Book ch 9.2]

University of Exeter

The ? operator

```rust
use std::num::ParseIntError;

fn double_parse(input: &str) -> Result<i32, ParseIntError> {
    let n: i32 = input.parse()?; // :i32  tells parse what type to produce
    Ok(n * 2)
}

fn main() {
    println!("{:?}", double_parse("5"));   // Ok(10)
    println!("{:?}", double_parse("abc")); // Err(invalid digit...)
}
```

- `?` after a `Result`: if `Ok`, unwrap the value; if `Err`, return the error early
- Replaces verbose `match` boilerplate for error propagation
- The function must return `Result` to use `?`
- [Book ch 9.2]

University of Exeter

Reading a file with `std::fs`

```rust
use std::fs;
use std::io;

fn read_file(path: &str) -> Result<String, io::Error> {
    let contents = fs::read_to_string(path)?;
    Ok(contents)
}

fn main() {
    match read_file("hello.txt") {
        Ok(text) => println!("{text}"),
        Err(e) => println!("Error: {e}"),
    }
}
```

- `fs::read_to_string`: reads an entire file into a `String`
- Returns `Result<String, io::Error>`: file might not exist or be unreadable
- `?` propagates the I/O error to the caller
- [Book ch 9.2, 12.2]

University of Exeter

Processing lines from a file

```rust
use std::fs;
use std::io;

fn count_lines(path: &str) -> Result<usize, io::Error> {
    let contents = fs::read_to_string(path)?;
    let mut count = 0;
    for line in contents.lines() {
        if !line.is_empty() {
            count += 1;
        }
    }
    Ok(count)
}

fn main() {
    match count_lines("hello.txt") {
        Ok(n) => println!("{n} non-empty lines"),
        Err(e) => println!("Error: {e}"),
    }
}
```

- `lines()` splits a `String` into an iterator of `&str` lines
- Use a `for` loop to process each line
- The file read and the line processing share the same `Result` return
- [Book ch 9.2, 12.2]

University of Exeter

Composing functions that return `Result`

```rust
use std::fs;
use std::io;

fn summarise_file(input: &str, output: &str) -> Result<(), io::Error> {
    let contents = fs::read_to_string(input)?;
    let line_count = contents.lines().count();
    let byte_count = contents.len();
    let summary = format!("{line_count} lines, {byte_count} bytes\n");
    fs::write(output, summary)?;
    Ok(())
}

fn main() {
    match summarise_file("hello.txt", "summary.txt") {
        Ok(()) => println!("Summary written"),
        Err(e) => println!("Error: {e}"),
    }
}
```

- Multiple `?` calls in one function: each can fail independently
- If any step fails, the function returns that error immediately
- `Ok(())` signals success with no return value (like `void` in C)
- [Book ch 9.2]

University of Exeter

`unwrap()` and why to avoid it in library code

```rust
use std::fs;

fn read_config() -> String {
    let contents = fs::read_to_string("config.txt").unwrap();
    contents
}

fn main() {
    let config = read_config();
    println!("{config}");
}
```

- `unwrap()`: extract the `Ok` value, panic on `Err`
- Convenient for quick prototyping and examples
- Avoid in library code: the caller has no chance to handle the error
- Rule: library functions should return `Result`, not call `unwrap()`
- [Book ch 9.3]

```
[gad212@mcrugcomp01 hello-rust]$ cargo run
   Compiling hello-rust v0.1.0 (/home/links/gad212/hello-rust)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.36s
     Running `target/debug/hello-rust`

thread 'main' panicked at src/main.rs:4:53:
called `Result::unwrap()` on an `Err` value: Os { code: 2, kind: NotFound, message: "No such file or directory" }
```

`main() -> Result` pattern

```rust
use std::error::Error;
use std::fs;
use std::io;

fn count_lines(path: &str) -> Result<usize, io::Error> {
    let contents = fs::read_to_string(path)?;
    let mut count = 0;
    for line in contents.lines() {
        if !line.is_empty() {
            count += 1;
        }
    }
    Ok(count)
}

fn main() -> Result<(), Box<dyn Error>> {
    let n = count_lines("hello.txt")?;
    println!("{n} non-empty lines");
    Ok(())
}
```

- `main` can return `Result<(), Box<dyn Error>>` instead of `()`
- `Box<dyn Error>` means "any kind of error" (covered with smart pointers later)
- This lets you use `?` directly in `main`
- On error, Rust prints the error and exits with a non-zero status code
- [Book ch 9.2]

Demo

University of Exeter

Recap

- `panic!`: unrecoverable errors, stops the program
- `Result<T, E>`: recoverable errors with `Ok` and `Err`
- `match`: handle `Result` locally (inspect or log the error)
- `?`: propagate errors to the caller
- `fs::read_to_string` + `lines()`: read and process files
- `unwrap()`: avoid in library code, return `Result` instead
- `main() -> Result<(), Box<dyn Error>>`: use `?` directly in main

- Required reading:
  - [Book ch 9.0-9.3]: Error handling
  - [Book ch 12.2]: Reading a file

University of Exeter